

Residual Multi-Tree Genetic Programming for Joint Adaptation of State Transition and Pheromone Update Rules in Ant Colony Optimization

First Author, Second Author, and Third Author

Abstract—Ant colony optimization depends on a state transition rule and a pheromone update rule. Both rules are usually designed by experts. Genetic programming can automate this design process. However, direct replacement of a complete rule can remove useful structure from the original algorithm. This study proposes residual multi-tree genetic programming for ant colony optimization, called RMTGP-ACO. It uses two strongly typed trees. The first tree applies a bounded correction to transition desirability. The second tree redistributes pheromone deposits under a fixed event budget. Zero output from both trees exactly recovers the original ACO. We implement the method for Ant System, Ant Colony System, and MAX-MIN Ant System. Training uses only uniform TSP100 instances. Each ACO variant uses three independent genetic programming runs. The selected programs are tested on uniform TSP50, TSP100, TSP500, and TSP1000. Full-F1 reduces the mean reference gap in all 12 variant-scale comparisons. The reduction ranges from 0.0736 to 7.8461 percentage points. Controlled pilot ablations show that the residual effect depends on the ACO variant. They do not show a stable benefit from the Full terminal set or the F1 function set. Both evolved trees affect the selected programs, but pair-specific coadaptation is not stable. All nine mean differences on the registered distribution-shift tests favor Full-F1. Under a total budget of 31 nodes, the two-tree method does not consistently outperform both one-tree controls. The 31/62-node capacity study remains ongoing. A fused CUDA evaluator gives a 14.626-fold single-GPU speedup over a 16-thread CPU reference on the registered benchmark. The results are a three-run pilot and are not confirmatory evidence.

Index Terms—Ant colony optimization, genetic programming, hyper-heuristic, multi-tree genetic programming, residual learning, traveling salesman problem.

I. INTRODUCTION

ANT colony optimization (ACO) constructs solutions from a pheromone model and problem-specific heuristic information [1], [2]. Its behavior depends on two central components. The state transition rule selects the next solution component. The pheromone update rule changes the sampling model after each iteration. These rules are effective, but they are usually fixed by human design.

Genetic programming (GP) can evolve executable rules [3], [4]. It can therefore reduce the need for manual formula design. Previous GP-ACO work evolved complete state transition rules [5]. That study showed that evolved rules can transfer across traveling salesman problem (TSP) sizes and distributions. It also showed that global terminals can be useful. However, a complete replacement changes the full

transition model. It does not preserve the original balance between pheromone and heuristic information. It can also produce unstable score scales.

Pheromone adaptation creates a second design problem. Earlier work used strongly typed GP to evolve pheromone strategies [6]. This direction is important, but joint transition and pheromone adaptation is difficult. The two rules act at different points in ACO. Their tensors also have different meanings and shapes. A shared untyped tree would allow invalid combinations. Two unrestricted trees would also receive more representational capacity than one tree.

We propose residual multi-tree genetic programming for ACO, or RMTGP-ACO. The method uses one transition tree and one pheromone tree. The trees have separate strong types. The transition tree multiplies the original desirability by a bounded residual factor. The pheromone tree redistributes a fixed deposit budget over the edges that the original ACO would reinforce. The method retains the main semantics of Ant System (AS), Ant Colony System (ACS), and MAX-MIN Ant System (MMAS) [1], [7], [8].

This study makes four technical contributions.

- It defines a bounded residual interface for ACO state transition.
- It defines an event-wise pheromone residual that preserves support and total deposit.
- It introduces a strongly typed two-tree representation with a shared node budget.
- It provides a fused GPU evaluator for the nested GP-ACO computation.

The current results form a three-run pilot. They answer whether the complete method improves the original ACO. They also provide controlled pilot evidence about the residual interface, tree roles, terminal and function profiles, and post-hoc coadaptation. The 31/62-node capacity analysis is still ongoing. We do not treat the current ablations as confirmatory causal evidence.

II. BACKGROUND AND PROBLEM DEFINITION

A. Euclidean Traveling Salesman Problem

Let $V = \{1, \dots, n\}$ be a set of cities. Each city i has a coordinate $c_i \in \mathbb{R}^2$. The distance between cities i and j is

$$d_{ij} = \|c_i - c_j\|_2. \quad (1)$$

A tour is a permutation $\pi = (\pi_1, \dots, \pi_n)$. Its length is

$$L(\pi) = \sum_{q=1}^{n-1} d_{\pi_q, \pi_{q+1}} + d_{\pi_n, \pi_1}. \quad (2)$$

The objective is to minimize $L(\pi)$.

Each dataset row contains a reference tour. We recompute its length in float64. The available files do not provide a global optimality certificate for every tour. We therefore use the term *reference length*, denoted by L_i^{ref} . The primary quality measure is

$$g_i(\theta) = 100 \frac{L_i(\theta) - L_i^{\text{ref}}}{L_i^{\text{ref}}}. \quad (3)$$

We call g_i the reference gap.

B. Baseline ACO State Transition

Let τ_{ij} be the pheromone value of edge (i, j) . Let

$$\eta_{ij} = \frac{1}{d_{ij} + \epsilon_d} \quad (4)$$

be its heuristic value. At construction step q , ant a is at city i . Its feasible set is $F_{a,q}$. The baseline desirability of city $j \in F_{a,q}$ is

$$s_{a ij}^0 = \tau_{ij}^\alpha \eta_{ij}^\beta. \quad (5)$$

AS and MMAS sample from the normalized score. ACS uses the pseudorandom proportional rule. It selects the maximum score with probability q_0 . Otherwise, it samples from the normalized score. All variants use a candidate list. If that list has no feasible city, the implementation uses the full feasible set.

C. Baseline Pheromone Update

AS evaporates all pheromone and reinforces the tours of all ants. For ant a , the baseline deposit on a tour edge is $1/L_a$. ACS applies its local update during construction. It then applies a global update to the global-best tour. MMAS reinforces one scheduled tour. The schedule uses iteration-best tours and restart-best tours. MMAS also applies dynamic pheromone bounds and restart logic.

Our implementation follows the algorithmic semantics of ACOTSP 1.03 [9]. All three variants use 32 ants and 500 iterations in this study. Local search is disabled.

D. Previous GP-ACO Baseline

The previous study evolved one complete state transition expression [5]. We denote a matched implementation of that approach by Legacy-GP. Its terminals are raw pheromone, raw distance, mean pheromone, mean distance, problem size, and feasible-set size. It uses arithmetic primitives and a protected division that returns one when the denominator is close to zero.

The present study retrains Legacy-GP under the current data, ACO budget, and random schedules. Published numerical results are not copied into the current paired tables. This rule prevents a comparison between incompatible budgets.

III. RESIDUAL MULTI-TREE GP-ACO

A. Overview

An RMTGP-ACO individual contains two programs:

$$\theta = (T_{\text{tr}}, T_{\text{ph}}). \quad (6)$$

The transition program acts during solution construction. The pheromone program acts after the colony tours are available. Fig. 1 shows the data flow.

B. Strongly Typed Representation

The implementation uses DEAP strongly typed GP [10], [11]. It defines two distinct Python marker classes: `TrField` and `PhField`. The two `PrimitiveSetTyped` root signatures are `TRANSITION: () -> TrField` and `PHEROMONE: () -> PhField`. Every subtree inside one tree has the same field type as its root. DEAP therefore rejects subtree exchange between the two semantic roles.

DEAP checks only the `TrField/PhField` marker during tree construction, crossover, and mutation. The shapes $[B, M, K]$ and $[B, R, n]$ are runtime contracts maintained by the evaluator. They are not additional DEAP static types. This separation allows B, K, R, n to change with the batch, ACO variant, and TSP size while preserving semantic type safety.

Each ACO task in B binds one program to one TSP instance. When one individual is evaluated, B is the instance batch. During fused population evaluation, B is the program-instance task batch. Usually, $K = \min(20, n - 1)$. Candidate-list fallback uses $K = n$. AS has $R = M = 32$. ACS has $R = 1$ and uses the global-best source tour. MMAS has $R = 1$ and retains its original reinforcement schedule.

The empty argument list in each root signature does not mean that a tree has no information. Each named terminal is a zero-arity typed leaf that is bound to the current ACO context at runtime. The evaluator constructs only terminals that are referenced by the program. The PyTorch backend can materialize the full fields. The fused CUDA and Numba backends compute the same logical values in flattened or compact form. The reported shapes are therefore logical tensor shapes. They do not require every terminal to be materialized at the same time.

If a fused call evaluates P deduplicated programs on I instances, then $B = PI$. For TSP100 with $M = 32$ and $K = 20$, the transition field is $[PI, 32, 20]$. The AS pheromone field is $[PI, 32, 100]$. The ACS and MMAS pheromone fields are $[PI, 1, 100]$.

The two trees share one fitness. They also share a total size limit:

$$N_{\text{tr}} + N_{\text{ph}} \leq 31. \quad (7)$$

An inactive zero tree has zero effective nodes. Each tree also has a maximum depth of five. Equation (7) is the active constraint in the main study. The two trees therefore share 31 effective nodes in total. The limit is not 31 nodes per tree. This rule prevents a two-tree individual from receiving twice the capacity of a one-tree individual.

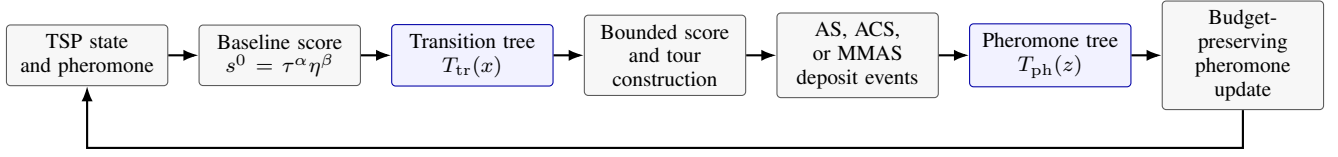


Fig. 1. RMTGP-ACO data flow. The blue boxes are evolved programs. The other boxes retain the semantics of the selected ACO variant.

TABLE I
INPUT, OUTPUT, AND EXECUTION LOCATION OF THE TWO TYPED GP TREES

Tree	Strong type and shape	Dimension meaning	Input–output contract	Execution point
Transition tree	TrField, $[B, M, K]$	B : ACO task batch; M : ants; K : current candidate width	Maps a dynamic selection context to one candidate-level output field	Every tour-construction step
Pheromone tree	PhField, $[B, R, n]$	R : reinforcement events; n : edges in one closed tour	Maps a dynamic reinforcement context to one event–edge output field	Reinforcement stage of every ACO iteration

TABLE II
RUNTIME CONTEXT AVAILABLE TO EACH TREE IN ONE CALL

Context	Raw information and shape	Meaning of one output element
Transition	Distance d , heuristic η , and pheromone τ : $[B, n, n]$; current city c : $[B, M]$; candidate city C and feasible mask F : $[B, M, K]$; baseline probability p^0 : $[B, M, K]$; construction step q , ACO iteration t , budget T , and stagnation counter $s[B]$	X_{bak}^{tr} is one terminal value for task b , ant a , and candidate slot k . C_{bak} maps that slot to a city.
Pheromone reinforcement	d, η, τ , and full nearest-neighbor rank: $[B, n, n]$; source tour π^{src} : $[B, R, n + 1]$; expanded endpoints u, v : $[B, R, n]$; source length: $[B, R]$; colony tours: $[B, M, n + 1]$; colony lengths: $[B, M]$; t, T , and $s[B]$	X_{bre}^{ph} is one terminal value for edge position e of reinforcement event r in task b .

C. Transition Residual

The transition tree receives the terminal field x_{aij} . Its raw output is

$$u_{aij} = T_{\text{tr}}(x_{aij}). \quad (8)$$

The output changes the baseline score in (5):

$$r_{aij} = \tanh(u_{aij}), \quad (9)$$

$$m_{aij} = 1 + \gamma_{\text{tr}} r_{aij}, \quad (10)$$

$$\tilde{s}_{aij} = s_{aij}^0 m_{aij}, \quad (11)$$

where $\gamma_{\text{tr}} = 1/3$. The final probability is

$$p_{aij} = \frac{\tilde{s}_{aij}}{\sum_{\ell \in F_{a,q}} \tilde{s}_{a\ell}}. \quad (12)$$

This interface has four useful properties. First, a zero tree gives $m_{aij} = 1$. It exactly recovers the baseline ACO. Second, $m_{aij} \in [2/3, 4/3]$. The score remains positive. Third, infeasible edges remain infeasible. Fourth, for two feasible cities j and k ,

$$\frac{m_{aij}}{m_{aik}} \in [1/2, 2]. \quad (13)$$

The tree can change a local preference, but it cannot create an unbounded score ratio.

D. Pheromone Residual

A deposit event is

$$r = (\pi_r, L_r, w_r). \quad (14)$$

π_r is a source tour. L_r is its length. w_r is the baseline weight. For each edge e in π_r , the baseline deposit is

$$D_{r,e}^0 = \frac{w_r}{L_r}. \quad (15)$$

The total event budget is

$$B_r = \sum_{e \in E(\pi_r)} D_{r,e}^0 = \frac{nw_r}{L_r}. \quad (16)$$

The pheromone tree produces

$$v_{r,e} = T_{\text{ph}}(z_{r,e}), \quad (17)$$

$$h_{r,e} = 1 + \gamma_{\text{ph}} \tanh(v_{r,e}), \quad (18)$$

where $\gamma_{\text{ph}} = 1/3$. The final deposit is

$$\tilde{D}_{r,e} = B_r \frac{D_{r,e}^0 h_{r,e}}{\sum_{f \in E(\pi_r)} D_{r,f}^0 h_{r,f}}. \quad (19)$$

The zero tree recovers the baseline deposit. The reinforced edge set is unchanged. The event budget is also conserved:

$$\sum_{e \in E(\pi_r)} \tilde{D}_{r,e} = B_r. \quad (20)$$

AS creates one event for each ant. ACS creates one global-best event. MMAS creates one event from its original update schedule. The residual only redistributes each event. It does not modify ACS local updates, MMAS bounds, or MMAS restart decisions.

E. Residual Strength

Both interfaces use the same residual strength, $\gamma = 1/3$. In general, the multiplier is

$$m(y) = 1 + \gamma \tanh(y), \quad 1 - \gamma \leq m(y) \leq 1 + \gamma. \quad (21)$$

Any $\gamma < 1$ preserves a positive multiplier. For $\gamma = 1/3$, one multiplier is bounded by $2/3$ and $4/3$. The ratio between the multipliers of two candidates is at most two. After probability or deposit-share normalization, an original share can change by at most a factor from $1/2$ to 2 . This range allows GP to change a relative preference while retaining the baseline rule as the dominant component.

The value $1/3$ was fixed before the test results were inspected. It was not selected on a test set. A value of 0.1 would allow only a weak correction. A value of 0.5 would allow a threefold relative correction between two candidates. We use the same value for both trees to remove one potential confounder. The current ablation does not treat γ as a factor. A future sensitivity study can compare 0.1 , $1/3$, and 0.5 .

F. Terminal Design Principles

The basic terminals in the previous GP-ACO study were pheromone concentration and distance [5]. Its extended configuration also included mean candidate pheromone, mean candidate distance, the number of cities, and the number of unvisited cities. Those terminals are suitable when GP must generate a complete transition score. We do not copy that set unchanged because the present trees learn residuals. The baseline score $s^0 = \tau^\alpha \eta^\beta$ already preserves the main absolute relation between pheromone and distance. The residual tree does not need to reconstruct that relation.

Table III maps information roles. It does not claim algebraic equivalence between the two terminal sets. We remove absolute means and raw city counts because they shift with the pheromone stage and problem size. The baseline ACO state still retains their effects. Our hypothesis is that relative inputs transfer more easily across sizes. Legacy-GP compares the two overall representations, but it cannot isolate one terminal.

The design follows four rules. First, every terminal must be available at decision time. Reference tours and reference lengths are never terminal inputs. Second, continuous inputs use relative values, ranks, or progress ratios when possible. Third, the set covers local edge quality, baseline confidence, search stage, and colony feedback. Fourth, the terminal count must support a controlled Core/Full ablation. We do not assume that Full is better. The Core/Full comparison tests the group-level increment. It does not prove that Core is minimal or identify the causal effect of one terminal.

G. Transition Terminal Set

Many raw ACO values depend on the problem size and iteration. The main terminal set uses bounded relative values. For a vector y and its active mask, define

$$\text{stdrel}(y) = \tanh\left(\frac{y - \text{mean}(y)}{\text{std}(y) + \epsilon}\right). \quad (22)$$

The mean and standard deviation are computed over the last active dimension. Masked entries receive zero.

RTau and REta describe the relative strength of the two baseline ACO signals. BaseConf describes confidence in one candidate. DistRank gives scale-insensitive local geometric order. These four terminals form Core. Entropy describes ambiguity of the complete choice distribution. ConstructProg, ACOProg, and Stagnation describe search context. They enlarge the search space and therefore belong only to Full.

H. Pheromone Terminal Set

Each value in Table V corresponds to one edge in one reinforcement event. The runtime context includes source tours $\pi^{\text{src}}[B, R, n+1]$, their lengths $L^{\text{src}}[B, R]$, colony tours $\pi^{\text{col}}[B, M, n+1]$, and colony lengths $L^{\text{col}}[B, M]$. Each source tour is expanded to endpoints $u, v[B, R, n]$. Every terminal therefore enters the tree as PhField $[B, R, n]$. An event scalar is broadcast over the edge axis. A task scalar is broadcast over both the event and edge axes.

EdgeEta, EdgeTau, and NNRank describe the reinforced edge. SourceQuality describes the route that produced that edge. These four terminals form Core. ColonyFreq, ACOProg, and Stagnation add colony and search-stage context. They belong only to Full.

Both primitive sets also contain the constants $-1, -0.5, 0, 0.5, 1$. They contain one ephemeral random constant sampled from $U[-1, 1]$. An ERC mutation adds Gaussian noise with standard deviation 0.1 . The result is clipped to $[-1, 1]$. The constants are also strongly typed terminals. A transition constant returns TrField. A pheromone constant returns PhField. The evaluator broadcasts a constant to $[B, M, K]$ or $[B, R, n]$. There is no untyped scalar terminal that can cross the two roles.

I. Function Sets and Numerical Protection

Table VI gives the two function profiles. All operations are element-wise. F1 extends F0. Protected division is

$$\text{PDIV}(x, y) = \frac{xy}{y^2 + 10^{-6}}. \quad (23)$$

This form is continuous at $y = 0$. After each primitive, NaN is replaced by zero. Positive and negative infinity are replaced by 10 and -10 . Every result is then clipped to $[-10, 10]$. The external residual interface applies another tanh.

Each function is registered twice. For example, transition ADD has signature $(\text{TrField}, \text{TrField}) \rightarrow \text{TrField}$. Pheromone ADD has signature $(\text{PhField}, \text{PhField}) \rightarrow \text{PhField}$. One call cannot mix the two types.

Legacy-GP uses ADD, SUB, MUL, NEG, and PDIV1. Its protected division returns x/y when $|y| > 10^{-6}$. It returns one otherwise. Matched-Replace uses the Full terminals and F1. Its raw transition output becomes a positive score through

$$s_{aij} = \text{softplus}(\text{clip}(u_{aij}, -20, 20)) + \epsilon. \quad (24)$$

TABLE III
RELATION BETWEEN THE PREVIOUS GP-ACO TERMINALS AND THE PRESENT DESIGN

Previous input	Corresponding information in this study	Design reason
Pheromone concentration and distance	RTau, REta, and the retained baseline score s^0	The ACO absolute score structure is retained. The tree observes relative strength inside the current candidate set.
Mean candidate pheromone and mean candidate distance	Within-group normalization of RTau/REta, plus BaseConf and Entropy	Location and scale enter the normalization. The latter terminals directly describe concentration of the baseline choice distribution.
Number of cities and number of unvisited cities	ConstructProg, DistRank, and NNRank	Ratios and ranks replace raw counts. Their ranges do not grow linearly with TSP size.
No pheromone-update-specific inputs	EdgeEta, EdgeTau, ColonyFreq, and SourceQuality	The second tree acts on reinforcement events. It requires edge quality, colony agreement, and source-tour quality.

TABLE IV
RUNTIME INPUTS, CALCULATION, AND LOGICAL TENSOR SHAPE OF TRANSITION TERMINALS

Terminal	Type; native shape $\rightarrow$ tree-input shape	Information read from the ACO state	Calculation and candidate-axis meaning	Profile
RTau	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\tau[B, n, n]$, current city [B, M], candidate city and feasible mask [B, M, K]	Gather pheromone from the current city to each candidate. Normalize $\log(\max(\tau, \epsilon))$ over the feasible K axis of each task-ant pair, then apply tanh.	Core, Full
REta	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\eta[B, n, n]$, current city [B, M], candidate city and feasible mask [B, M, K]	Gather the heuristic value for each candidate. Apply log normalization over the feasible K axis and then tanh.	Core, Full
BaseConf	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\tau, \eta, \alpha, \beta$, feasible mask, and $p^0[B, M, K]$ from $s^0 = \tau^\alpha \eta^\beta$	For feasible count k , compute $\tanh[\log(p^0 + \epsilon) + \log k]$ for each candidate. A uniform distribution maps to zero.	Core, Full
DistRank	TrField; [B, M, K] $\rightarrow$ [B, M, K]	Distance matrix $d[B, n, n]$, current city, candidate city, and feasible mask	Rank candidates by distance within each feasible K set. Map rank r to $1 - 2(r - 1)/(k - 1)$. Use zero when $k = 1$.	Core, Full
Entropy	TrField; [B, M, 1] $\rightarrow$ [B, M, K]	Baseline probability $p^0[B, M, K]$, feasible mask, and feasible count $k[B, M, 1]$	Compute $2H(p^0)/\log k - 1$ over K, then broadcast to all candidates. Use -1 when $k = 1$.	Full
ConstructProg	TrField; [1] $\rightarrow$ [B, M, K]	Current construction step q and city count n	Compute $2q/\max(n - 1, 1) - 1$, then broadcast over B, M, K.	Full
ACOProg	TrField; [1] $\rightarrow$ [B, M, K]	Current ACO iteration t and total iterations T	Compute $2(t - 1)/\max(T - 1, 1) - 1$, then broadcast to all ants and candidates.	Full
Stagnation	TrField; [B, 1, 1] $\rightarrow$ [B, M, K]	Task-level non-improvement count $s_t[B]$ and T	Compute $2\min(s_t/T, 1) - 1$, then broadcast over ants and candidates.	Full

F0 can form smooth low-order combinations. F1 adds simple piecewise rules through MIN, MAX, and ABS. We omit exponential, logarithmic, and trigonometric primitives. The terminals already include log transformations and bounded normalization. The residual interface also applies tanh. Adding these functions would enlarge the equivalent-expression space and increase numerical risk. We also omit explicit conditional branches. MIN and MAX provide simple gating and are easier to fuse. The F0/F1 factor tests whether the additional functions are necessary.

J. Evolutionary Search

The initial joint population contains four modes. Ten percent are exact baseline sentinels. Twenty percent activate only the transition tree. Twenty percent activate only the pheromone tree. The remaining 50 percent activate both trees. Active trees use ramped half-and-half initialization with depths from two to four.

Elitism keeps the best ten individuals. The remaining offspring use crossover with probability 0.80, mutation with probability 0.15, and reproduction with probability 0.05. Tournament size is four. Crossover selects one active role and applies typed one-point crossover within that role. Mutation selects one active role. It uses subtree mutation with probability 0.50, node replacement with probability 0.30, and ERC perturbation

with probability 0.20. An offspring that violates the depth or node limit is replaced by its valid parent.

Structural hashes avoid repeated evaluation within one generation. The GPU evaluator also groups programs with identical compiled behavior. These operations do not change the fitness.

For a training set $\mathcal{I}$ and ACO seed set $\mathcal{S}$, fitness is

$$F(\theta) = \frac{1}{|\mathcal{I}||\mathcal{S}|} \sum_{i \in \mathcal{I}} \sum_{s \in \mathcal{S}} g_{i,s}(\theta). \quad (25)$$

The objective minimizes the absolute reference gap. It does not minimize a baseline-relative value. It contains no parsimony penalty. Program size is used only to break defined ties. For reporting, we separately define

$$\Delta g = g_{\text{method}} - g_{\text{ACO}}. \quad (26)$$

A negative Δg favors the learned method.

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

Table VII lists the seven research questions and the available evidence. RQ1, RQ2, and RQ4–RQ6 have complete three-run pilot quality results. The 31-node part of RQ3 is complete. The 31/62-node capacity study is ongoing. The fused training

TABLE V
RUNTIME INPUTS, CALCULATION, AND LOGICAL TENSOR SHAPE OF PHEROMONE TERMINALS

Terminal	Type; native shape $\rightarrow$ tree-input shape	Information read from the ACO state	Calculation and event-edge meaning	Profile
EdgeEta	PhField; $[B, R, n] \rightarrow [B, R, n]$	$\eta[B, n, n]$, endpoints $u, v[B, R, n]$, and candidate-neighborhood mean $\log \eta[B, n]$ for each city	Subtract the mean endpoint neighborhood value from edge $\log \eta_{uv}$. Normalize over the n edges of each task-event pair and apply tanh.	Core, Full
EdgeTau	PhField; $[B, R, n] \rightarrow [B, R, n]$	$\tau[B, n, n]$ and endpoints $u, v[B, R, n]$	Gather τ_{uv} for every source edge. Normalize $\log(\max(\tau_{uv}, \epsilon))$ over the n edges of each event and apply tanh.	Core, Full
NNRank	PhField; $[B, R, n] \rightarrow [B, R, n]$	Precomputed full nearest-neighbor ranks $[B, n, n]$ and endpoints $u, v[B, R, n]$	Map the directed ranks $u \rightarrow v$ and $v \rightarrow u$ by $1 - 2(r - 1) / \max(n - 2, 1)$, then average the two directions.	Core, Full
ColonyFreq	PhField; $[B, R, n] \rightarrow [B, R, n]$	Current colony tours $\pi^{\text{col}}[B, M, n + 1]$ and source endpoints u, v	Ignore direction and count how many of the M colony tours contain each source edge. Compute $2c_e/M - 1$.	Full
SourceQuality	PhField; $[B, R, 1] \rightarrow [B, R, n]$	Source length $L^{\text{src}}[B, R]$ and colony lengths $L^{\text{col}}[B, M]$	Compute colony mean $\bar{L}$ and population standard deviation σ_L over M . Use $\tanh[(\bar{L} - L_r^{\text{src}})/(\sigma_L + \epsilon)]$, then broadcast over the n edges.	Core, Full
ACOProg	PhField; $[1] \rightarrow [B, R, n]$	Current ACO iteration t and total iterations T	Compute $2(t - 1) / \max(T - 1, 1) - 1$, then broadcast to all events and edges.	Full
Stagnation	PhField; $[B, 1, 1] \rightarrow [B, R, n]$	Task-level non-improvement count $s_t[B]$ and T	Compute $2 \min(s_t/T, 1) - 1$, then broadcast over events and edges.	Full

TABLE VI
GP FUNCTION SET AND STRONGLY TYPED INPUT-OUTPUT SIGNATURES

Function	Calculation	Input type	Output type	Profile
ADD	$x + y$	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F0, F1
SUB	$x - y$	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F0, F1
MUL	xy	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F0, F1
PDIV	$xy/(y^2 + 10^{-6})$	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F0, F1
NEG	$-x$	TrField or PhField	TrField or PhField	F0, F1
MIN	$\min(x, y)$	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F1
MAX	$\max(x, y)$	TrField $\times$ TrField or PhField $\times$ PhField	TrField or PhField	F1
ABS	$ x $	TrField or PhField	TrField or PhField	F1

benchmark in RQ7 is complete, while some isolated single-program timing cells are ongoing.

B. Data

The training pool contains 1,280,000 uniform TSP100 instances. It consists of ten files with 128,000 instances each. A frozen schedule selects 32 different instances per generation. One GP run therefore sees $32 \times 50 = 1,600$ different training instances. The full pool is much larger than the consumed subset. This design supports independent schedules for additional GP runs.

The validation file contains 1,280 uniform TSP100 instances. Each GP run uses 32 instances for selection and 32 disjoint instances for the non-inferiority gate. The completed test sets contain 1,280 uniform TSP50 instances, 1,280 uniform TSP100 instances, 128 uniform TSP500 instances, and 128 uniform TSP1000 instances. TSP1000 is a locked-model supplementary extrapolation set. It is not used during evolution or model selection.

The registered distribution tests contain 128 clustered TSP500 instances, 128 Gaussian TSP500 instances, and 42

TSPLIB instances with $n \leq 500$ [12]. They are used only for the locked Full-F1 champions. No program is selected again on these partitions. All files are identified by a manifest and a SHA-256 checksum. Train, validation, and test coordinates remain separated. Reference-tour edges are never used as GP terminals.

C. Compared Methods and Ablations

Table IX maps each method to its scientific purpose. Full-F1 is the proposed main configuration. Core and Full refer to the terminal sets in Tables IV and V. F0 and F1 refer to Table VI.

RQ2 uses TR-RGP minus Matched-Replace as its primary estimand. Legacy-GP is a secondary methodological baseline. RQ3 compares Full-F1 with independently trained TR-RGP and PH-RGP. All three methods first use a total node budget of 31. A separate capacity study compares budgets of 31 and 62. RQ4 uses a 2×2 factorial design. It estimates the terminal main effect, the function main effect, and their interaction. RQ5 drops each tree and shuffles independently evolved tree pairs. The post-hoc interventions complement the independent one-tree controls. They do not replace them.

TABLE VII
RESEARCH QUESTIONS, PRIMARY COMPARISONS, AND CURRENT STATUS

Question	Research question	Primary comparison	Status
RQ1	Does Full-F1 reduce the reference gap of the original ACO?	Full-F1 versus the corresponding AS, ACS, or MMAS baseline	Complete
RQ2	Is residual transition adaptation better than complete transition replacement?	TR-RGP versus Matched-Replace	Complete
RQ3	Under the same total node budget, is a two-tree method better than a one-tree method?	Full-F1 versus independently trained TR-RGP and PH-RGP	B31 complete; capacity ongoing
RQ4	Does the improvement depend on Full terminals or F1 functions?	Core/Full $\times$ F0/F1 factorial experiment	Complete
RQ5	Is there joint coadaptation between the two trees?	Tree deletion and cross-run tree-pair shuffling	Complete
RQ6	Does a TSP100-trained program transfer across sizes and distributions?	TSP50/500/1000, cluster, Gaussian, and TSPLIB	Complete
RQ7	How much acceleration does the fused implementation provide?	CPU8, CPU16, one GPU, two-GPU shard, and parallel campaign	Training complete; deployment timing ongoing

Algorithm 1 One RMTGP-ACO simulation

Require: TSP instance, ACO variant, T_{tr} , T_{ph}

- 1: Initialize pheromone and variant-specific search state
- 2: **for** $t = 1, \dots, T$ **do**
- 3: **for** each construction step q **do**
- 4: Build the active candidate set
- 5: Compute baseline score $s^0 = \tau^\alpha \eta^\beta$
- 6: Build only the terminals used by T_{tr}
- 7: Apply the bounded residual in (11)
- 8: Select cities with the original variant rule
- 9: Apply the ACS local update when required
- 10: **end for**
- 11: Compute all tour lengths and update best states
- 12: Build AS, ACS, or MMAS deposit events
- 13: Build only the terminals used by T_{ph}
- 14: Apply the budget residual in (19)
- 15: Apply variant evaporation, bounds, and restart logic
- 16: **end for**
- 17: **return** Best tour; recompute its length in CPU float64

Algorithm 2 GP training and champion selection

Require: Frozen training and validation schedules

- 1: Load or create immutable baseline ACO archives
- 2: Initialize 100 typed two-tree individuals
- 3: **for** $h = 1, \dots, 50$ **do**
- 4: Load 32 scheduled TSP100 training instances
- 5: Use one shared ACO seed for all individuals
- 6: Evaluate unique programs and assign (25)
- 7: Save the top five every five generations
- 8: Create the next population with typed operators
- 9: **end for**
- 10: Screen unique checkpoints on 32 selection instances with one seed
- 11: Re-evaluate the best five with three paired seeds
- 12: Find the minimum mean gap
- 13: Within 0.01 percentage points, select the smaller program
- 14: Audit this candidate on 32 disjoint gate instances
- 15: **return** One selected candidate and one gate-deployed policy

TABLE VIII
DATA USED BY THE TSP100 PILOT

Role	Partition	Instances
Training pool	TSP100 uniform	1,280,000
Per GP run	32×50 TSP100	1,600
Validation	Selection / gate	32/32
Test	TSP50 uniform	1,280
Test	TSP100 uniform	1,280
Test	TSP500 uniform	128
Test	TSP1000 uniform	128
OOD test	TSP500 cluster / Gaussian	128/128
OOD test	TSPLIB, $n \leq 500$	42

D. ACO and GP Parameters

Table X gives the common parameters. AS uses $\rho = 0.5$. ACS uses $\rho = 0.1$, $q_0 = 0.9$, and $\xi = 0.1$. MMAS uses $\rho = 0.02$. Its update period is 25. Its p_{best} value is 0.05. Its branch check period is 100. Its branch parameter is 0.05. Its branch threshold is 1.00001. Its restart stagnation limit is 250.

ACS applies synchronous local updates.

The GP root seeds are 1001–1003 for AS, 2001–2003 for ACS, and 3001–3003 for MMAS. Final test seeds are derived from a separate root seed of 9001. They are independent of the GP seeds. All methods within a paired comparison share the same instances and ACO seeds.

E. Champion Selection and Reported Policy

Each generation uses one ACO seed. All individuals share that seed and the same 32 instances. This pairing reduces within-generation noise. The baseline ACO values are pre-computed and stored in immutable archives.

The validation procedure first screens checkpoints with one seed. It then evaluates the five best checkpoints with three seeds. It selects the smallest tree among candidates within 0.01 percentage points of the best mean gap. Only this selected individual enters the independent gate. The gate

TABLE IX
METHODS, CONTROLLED DIFFERENCES, AND CURRENT RESULT STATUS

Method	Transition integration	Pheromone integration	Terminal/function profile	Main purpose	Status
Baseline ACO	None	None	None	Original AS, ACS, or MMAS	Complete
Legacy-GP	Full replacement	None	Legacy	Prior GP-ACO structure under the current budget	Complete
Matched-Replace	Full replacement	None	Full/F1	Replacement control for RQ2	Complete
TR-RGP	Residual	None	Full/F1	Independent transition-only control	Complete
PH-RGP	None	Budget residual	Full/F1	Independent pheromone-only control	Complete
RMTGP Core-F0	Residual	Budget residual	Core/F0	Factorial terminal/function control	Complete
RMTGP Core-F1	Residual	Budget residual	Core/F1	Factorial terminal/function control	Complete
RMTGP Full-F0	Residual	Budget residual	Full/F0	Factorial terminal/function control	Complete
RMTGP Full-F1	Residual	Budget residual	Full/F1	Proposed main method	Complete
Drop and shuffle	Post-hoc intervention	Post-hoc intervention	Frozen Full/F1 trees	Coadaptation analysis for RQ5	Complete

TABLE X
COMMON EXPERIMENTAL PARAMETERS

Parameter	Value
ACO ants	32
ACO iterations	500
Candidate-list size	20
α, β	1, 2
γ_{tr}, γ_{ph}	1/3, 1/3
Local search	Disabled
GP population	100
GP generations	50
Crossover / mutation / reproduction	0.80/0.15/0.05
Elites / tournament size	10/4
Initial depth / maximum depth	2-4/5
Total effective node budget	31
Checkpoint interval / retained count	5/5
Training instances per generation	32
Selection / gate instances	32/32
Screening / final validation seeds	1/3
Test ACO seeds per instance	3
Independent GP runs per variant	3
CPU threads / process count	16/1
CUDA block threads	32
Search / final score precision	FP32 / FP64

accepts a candidate when the one-sided 95% upper bound of its validation delta is at most 0.10 percentage points.

The main scientific analysis uses the selected candidate from each GP run. It retains all three GP runs. It does not select the best GP seed. The gate-deployed policy is a separate operational quantity. It falls back to baseline ACO when a candidate fails the gate.

F. Statistical Analysis

The primary response is the reference gap in (3). We also record worst-decile conditional value at risk, anytime gap area under the curve, best iteration, wall time, and win/tie/loss rates. A win means that the learned method has a smaller gap than its paired baseline.

The three ACO seeds are first averaged within an instance. The main statistical block is then GP-run by instance. We use a paired Wilcoxon signed-rank test [13]. Holm correction is applied within each ACO variant and RQ family [14]. We

also use a hierarchical bootstrap [15]. It resamples GP runs, then instances, then ACO seeds. Each interval uses 10,000 replicates.

Only three GP runs are available in this pilot. This count is too small for a final claim about evolutionary-run uncertainty. The confidence intervals and tests are therefore descriptive. A confirmatory study will use 30 independent GP runs.

G. Evolution Curves and Final-Test Protocol

The training curve does not use the median fitness of the population. For each independent GP run and generation, we first select the individual with minimum fitness. Program size breaks an exact fitness tie. The training point is the mean reference gap of that generation-best individual on the 32 new training instances. Because every generation uses a different batch, short-term changes are not strict estimates of generalization.

The validation point evaluates the same generation-best individual on the fixed 32 selection instances and the fixed screening seed. It is not the population median or the best-so-far validation value. For each ACO variant, the plotted line is the median of the three independent GP runs at the same generation. The band is their minimum and maximum. The baseline line uses the same fixed validation instances and seed.

The final test does not use the last-generation individual directly. The candidate pool contains the top five individuals from every fifth generation and the top five from the last generation. Duplicate structures are removed. Screening, three-seed reevaluation, and the independent gate select one champion for each GP run. All three champions enter the scientific test. The test sets are used only for final measurement.

V. EFFICIENT IMPLEMENTATION

A. Batched Evaluation Architecture

The nested fitness is expensive. One population contains 100 individuals. Each individual invokes ACO on 32 instances. Each ACO invocation constructs 32×500 tours. A direct Python loop over individuals, instances, ants, and iterations is not practical.

TABLE XI
FORMAL ACCELERATOR BENCHMARK

Mode	Time (s)	Tours/s	CPU16 ratio
CPU8	397.422	101,776	0.766
CPU16	304.325	132,911	1.000
GPU0	20.807	1,943,957	14.626
GPU1	20.838	1,941,110	14.605
Dual shard	12.776	3,165,928	23.820
Campaign [†]	21.101	3,833,840	28.9

[†]Two concurrent workloads; ratio uses aggregate throughput.

The implementation uses NumPy, Numba, PyTorch, DEAP, and CuPy [11], [16]–[19]. The CPU backend forms a population-by-instance task matrix. Numba compiles the inner ACO loops. The GPU backend goes further. It compiles each typed GP tree to a compact postfix program. It removes structurally and behaviorally duplicate programs. It then forms a program-by-instance task grid.

One CUDA block owns one program–instance task. Thirty-two threads match the 32 ants. The block performs all 500 ACO iterations. It evaluates both GP programs inside the kernel. It also performs candidate fallback, ACS local-update multiplicities, MMAS bounds, and MMAS restart logic. Problem tensors and terminal-independent features remain resident on the device. A counter-based random generator makes task partitioning deterministic.

The GPU searches in FP32. It returns the best tour. The host recomputes that tour length from the original float64 distance matrix. The final fitness and all paper tables therefore use CPU FP64 tour scoring.

B. Parallel Scheduling

One GP run uses one process and one visible GPU. Population-level and instance-level work are fused inside the kernel. This design avoids multiple Python processes competing for one GPU. Independent GP runs are the main multi-GPU unit. They are assigned to different devices. This campaign mode maximizes aggregate throughput.

A dual-shard mode can split one generation over two GPUs. It reduces the latency of one run. Its scaling is weaker because each shard is smaller. The registered scheduler therefore uses independent runs when enough runs are available. CPU16 remains the deterministic fallback and the FP64 audit backend.

C. Formal Acceleration Benchmark

The formal benchmark used an Intel Xeon W-2245 CPU and two 20-GiB NVIDIA RTX 4000 Ada Generation GPUs. The workload used ACS, 100 requested individuals, 79 behaviorally unique programs, 16 TSP50 instances, 16 TSP100 instances, 32 ants, and 500 ACO iterations. One workload constructed 40,448,000 tours. Each mode used one cold run and three warm runs. Table XI reports the warm median.

Single-run dual-GPU scaling is $20.807/12.776 = 1.629$. Concurrent campaign scaling is $2(20.807)/21.101 = 1.972$. The first value is below the registered 1.7 scaling gate. The second value is above the registered 1.8 campaign gate. This result supports task-level multi-GPU scheduling.

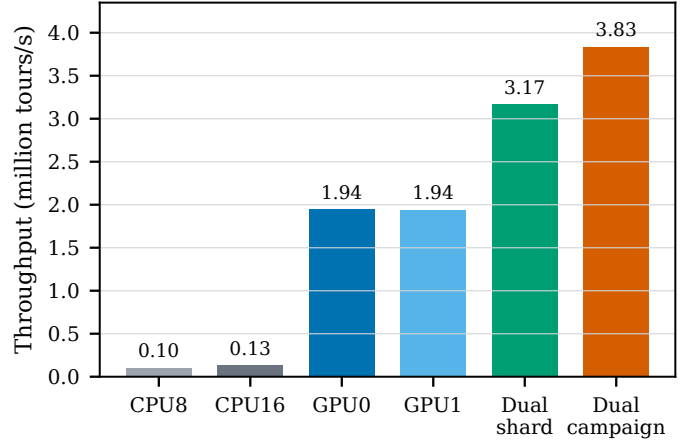


Fig. 2. Throughput on the formal accelerator benchmark. Campaign throughput is aggregated over two concurrent workloads.

TABLE XII
OBSERVED GENERATION TIME ACROSS THREE GP RUNS

Variant	Mean $\pm$ SD (s)	Run range (s)
AS	34.50 $\pm$ 2.97	31.13–36.78
ACS	26.88 $\pm$ 4.43	21.97–30.57
MMAS	22.14 $\pm$ 3.34	18.31–24.41

Average GPU utilization was about 99%. Peak device memory was about 286 MiB. About 99.8% of warm time was inside the CUDA kernel. The workload was compute-bound. Additional memory caching is therefore unlikely to be the main source of future speedup.

The numerical quality audit used TSP50 and TSP100. It used 128 instances per size and three paired ACO seeds. Let $\Delta_{\text{num}} = g_{\text{GPU}} - g_{\text{CPU}}$. The pooled mean was -0.003445 percentage points. The one-sided 95% upper bound was 0.000868 percentage points. This value is below the registered 0.10 threshold.

The observed mean generation times are shown in Table XII. They include all unique population evaluations for one generation. They do not include final test evaluation.

VI. RESULTS AND ANALYSIS

A. RQ1: Full-F1 Versus Baseline ACO

Table XIII reports the completed selected-candidate results. The baseline column contains the original ACO gap. The Full-F1 column contains the learned-method gap. The confidence interval is for Δg . Negative values favor Full-F1.

All 12 mean differences are negative. All 12 bootstrap intervals are below zero. The smallest observed reduction is 0.0736 percentage points for ACS on TSP50. The largest reduction is 7.8461 percentage points for AS on TSP500. The Holm-adjusted Wilcoxon values are at most 9.004×10^{-17} . These test values describe the current blocks. They do not replace replication over more GP runs.

AS improves from 1.8803% to 1.7479% on TSP50. It improves from 5.8952% to 4.2154% on TSP100. Its reductions

TABLE XIII

SELECTED-CANDIDATE RESULTS ON COMPLETED UNIFORM TEST SETS. CI DENOTES THE HIERARCHICAL BOOTSTRAP 95% INTERVAL OF Δg . W/T/L IS REPORTED IN PERCENT.

ACO	n	Baseline gap (%)	Full-F1 gap (%)	Δg (pp)	95% CI (pp)	W/T/L
AS	50	1.8803	1.7479	-0.1324	[-0.2076, -0.0718]	54.7/2.8/42.4
AS	100	5.8952	4.2154	-1.6799	[-1.9680, -1.4739]	92.1/0.0/7.9
AS	500	22.9874	15.1413	-7.8461	[-8.3979, -7.3727]	100.0/0.0/0.0
AS	1000	30.4224	23.1088	-7.3135	[-8.5940, -6.1418]	100.0/0.0/0.0
ACS	50	0.7326	0.6589	-0.0736	[-0.1050, -0.0432]	46.6/17.6/35.9
ACS	100	1.9331	1.6412	-0.2918	[-0.3954, -0.2091]	64.7/0.0/35.3
ACS	500	16.1641	12.9399	-3.2242	[-4.5698, -1.3932]	93.8/0.0/6.2
ACS	1000	21.6293	19.2041	-2.4252	[-3.3961, -0.8317]	94.8/0.0/5.2
MMAS	50	0.5941	0.4727	-0.1215	[-0.1549, -0.0820]	52.3/22.4/25.2
MMAS	100	4.1088	1.8106	-2.2982	[-2.4240, -2.1223]	99.2/0.0/0.8
MMAS	500	20.6291	14.1769	-6.4522	[-8.1652, -4.7067]	100.0/0.0/0.0
MMAS	1000	25.4885	18.8571	-6.6314	[-8.3746, -4.8120]	100.0/0.0/0.0

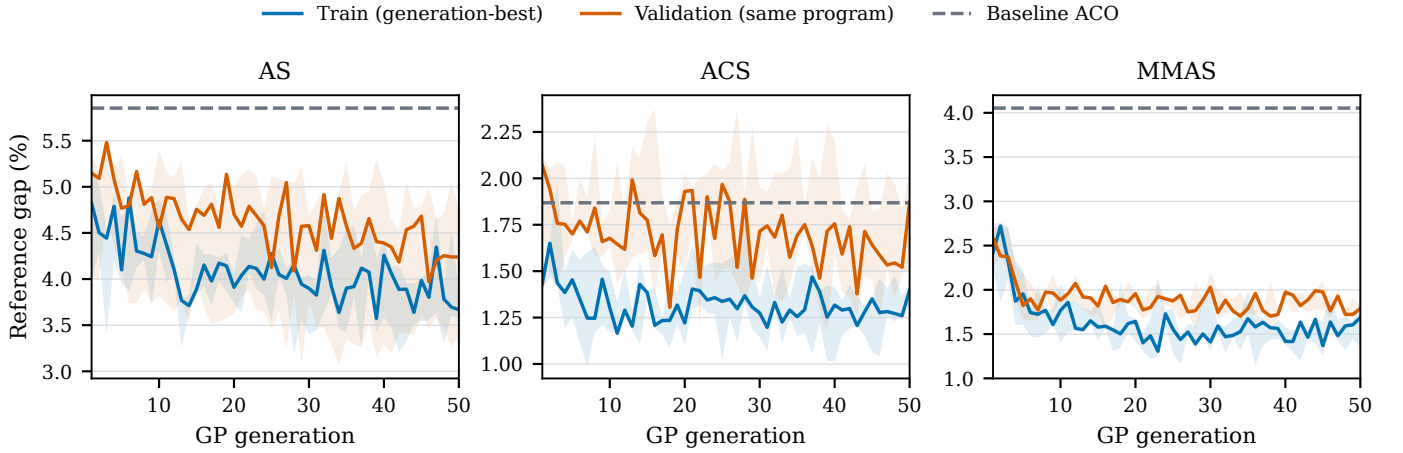


Fig. 3. TSP100 training and validation curves. For each run, a training point is the generation-best individual on that generation's new batch. The validation point evaluates the same individual on the fixed selection set. Lines are medians over three independent GP runs. Bands show their minimum and maximum.

are larger on TSP500 and TSP1000. They are 7.8461 and 7.3135 percentage points.

ACS has smaller gains at the two smaller sizes. Its reduction is 0.2918 percentage points on TSP100. It reaches 3.2242 percentage points on TSP500. One of the three ACS candidates did not pass the independent deployment gate. The selected-candidate table retains that run because the analysis must not remove an unfavorable GP replicate. The operational gate-deployed result is therefore weaker than the selected-candidate result for ACS.

MMAS improves from 4.1088% to 1.8106% on TSP100. Its reductions are 6.4522 and 6.6314 percentage points on TSP500 and TSP1000. Its TSP100 win rate is 99.2%. Its TSP500 and TSP1000 win rates are 100% in the current blocks.

Fig. 3 shows noisy training curves. This behavior is expected because every generation uses a new batch. The validation curves are also not monotonic. The selected checkpoint can therefore differ from the last generation. The independent selection procedure is necessary.

Fig. 5 uses the same validation protocol. The curves diagnose evolution only. Final comparisons use independently selected champions and locked test sets. A zero tree is a

valid individual. A zero-delta plateau for Legacy-GP therefore means that a zero tree won that generation; it is not missing data.

Answer to RQ1: Full-F1 has a lower mean reference gap than the corresponding original ACO in every completed pilot comparison. This is evidence for effectiveness. It is not yet evidence that the residual interface, two-tree structure, or Full/F1 primitive profile is the cause.

B. RQ2: Residual Versus Full Replacement

Table XIV and Fig. 6 give the direct RQ2 estimand. TR-RGP and Matched-Replace both use one transition tree. They share the Full terminals, F1 functions, 31-node budget, training data, and random schedule. Only the integration interface changes.

Both AS intervals are below zero. The estimates are -0.6058 and -2.5584 percentage points. Both ACS intervals are above zero. The estimates are $+0.3114$ and $+5.4303$ percentage points. Both MMAS means are negative, at -0.6795 and -1.7058 , but both intervals cross zero. Thus, two of six contexts support residual integration. Two support complete replacement. Two favor residual integration in direction only.

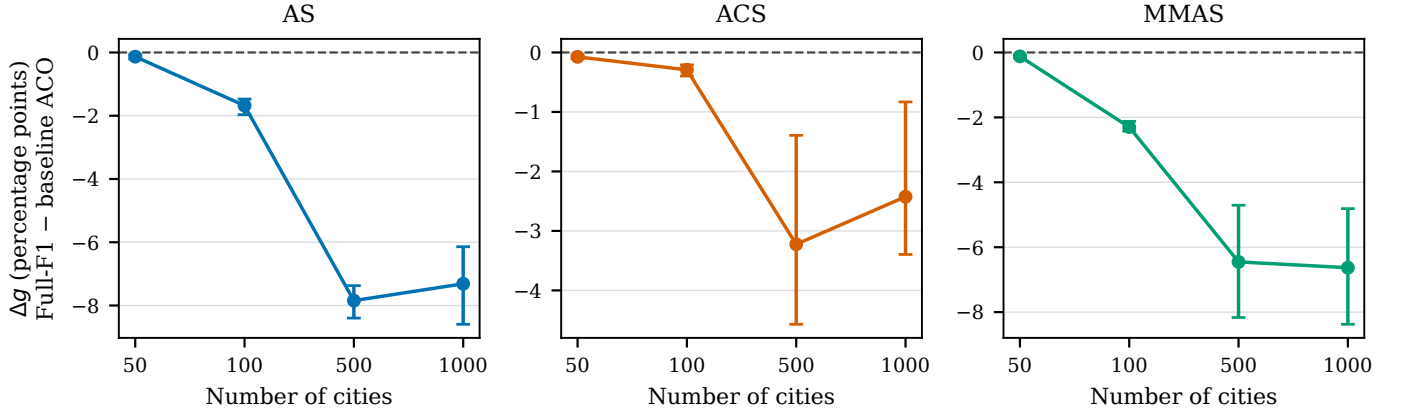


Fig. 4. Full-F1 minus baseline ACO reference gap. Error bars are hierarchical bootstrap 95% intervals. Negative values favor Full-F1.

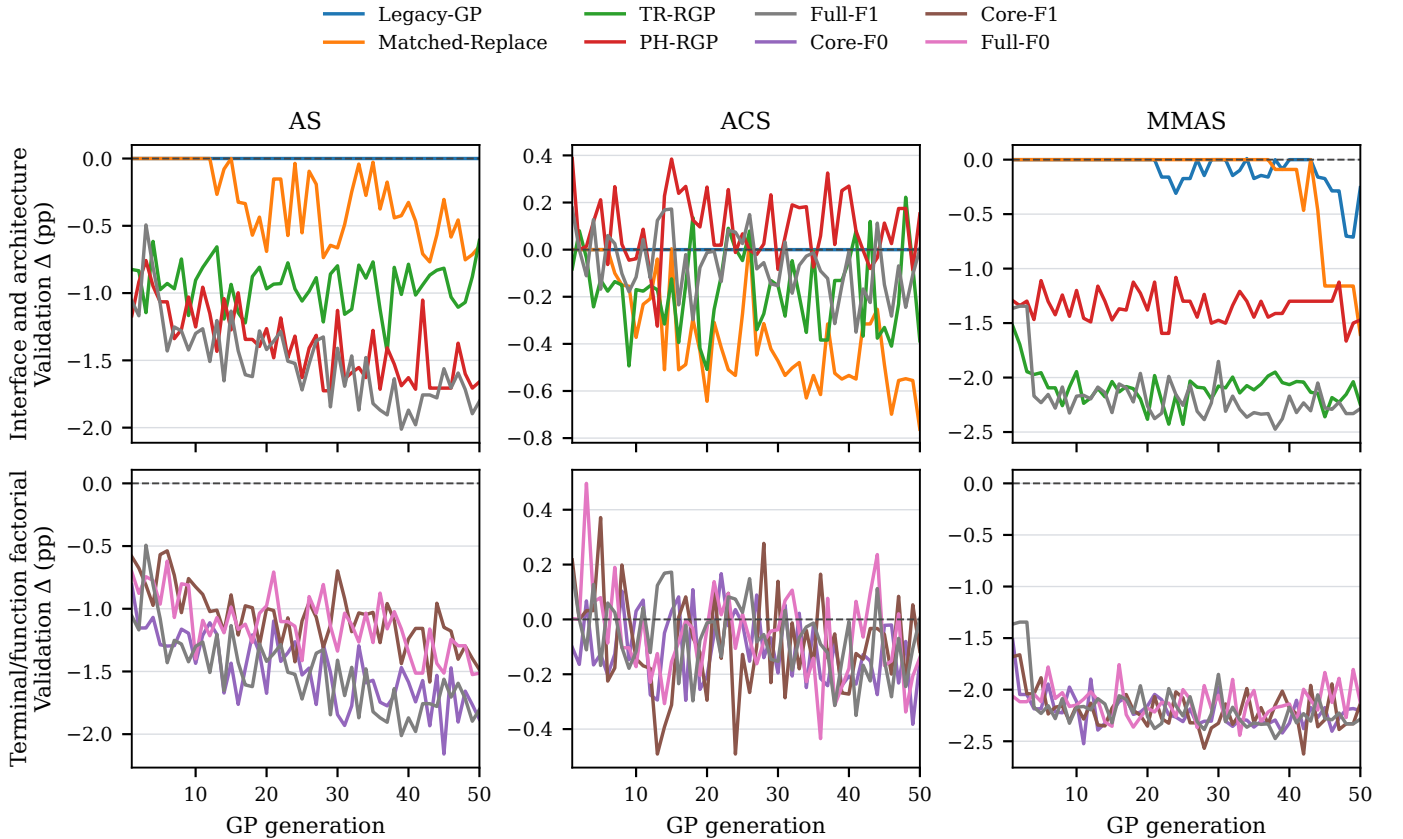


Fig. 5. Validation-delta curves for all ablation methods on the fixed selection set. Each line is the median of three GP runs at the same generation. The upper row compares integration interfaces and tree structures. The lower row compares Core/Full $\times$ F0/F1. Negative values are below the original ACO.

MMAS pheromone bounds and the residual interface both limit a deviation. They do not act on the same variable. MMAS clips updated pheromone. RQ2 changes the candidate transition score. The two mechanisms should therefore not be treated as equivalent. The present MMAS intervals also do not show a clear residual advantage.

Legacy-GP changes the terminal and function sets together with the interface. It is an auxiliary prior-work baseline and cannot isolate the residual effect.

Answer to RQ2: The pilot does not support a universal residual advantage. Residual integration is better for AS.

Complete replacement is better for ACS. The MMAS evidence is uncertain.

C. RQ3: Two Trees Versus One Tree

1) *Architecture at a Fixed 31-Node Budget:* Table XV and Fig. 7 compare Full-F1, TR-RGP, and PH-RGP. All three methods are trained independently. All three share 31 total effective nodes. The two-tree method divides that total between its two trees.

Full-F1 has two intervals below zero relative to TR-RGP. The other four intervals cross zero. No interval clearly favors

TABLE XIV

PAIRED RESIDUAL-VERSUS-REPLACEMENT DIFFERENCES. EACH CELL IS THE MEAN DIFFERENCE [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS. NEGATIVE VALUES FAVOR THE RESIDUAL INTERFACE.

ACO	Test	TR-RGP — Matched-Replace
AS	TSP100-U	-0.606 [-0.853, -0.413]
AS	TSP500-U	-2.558 [-3.119, -1.720]
ACS	TSP100-U	+0.311 [+0.216, +0.436]
ACS	TSP500-U	+5.430 [+3.726, +6.602]
MMAS	TSP100-U	-0.679 [-1.224, +0.291]
MMAS	TSP500-U	-1.706 [-3.790, +0.452]

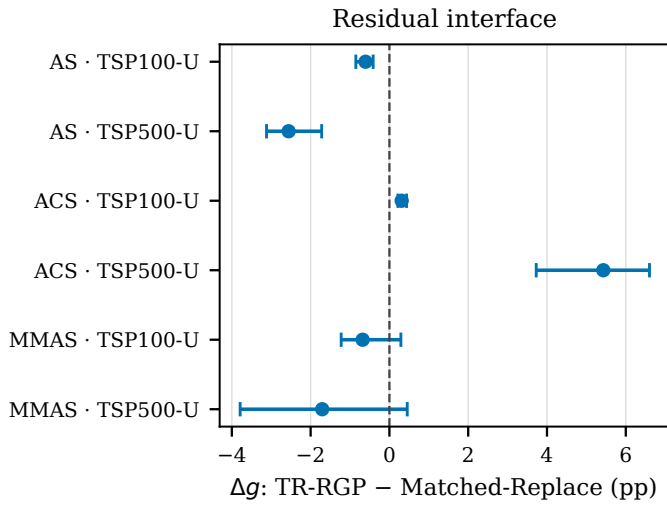


Fig. 6. Paired residual-versus-replacement differences. Error bars are 95% hierarchical bootstrap intervals. Negative values favor residual integration.

TABLE XV

PAIRED DIFFERENCES BETWEEN THE TWO-TREE METHOD AND INDEPENDENTLY TRAINED ONE-TREE METHODS UNDER A TOTAL 31-NODE BUDGET. EACH CELL IS THE MEAN DIFFERENCE [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS. NEGATIVE VALUES FAVOR THE TWO-TREE METHOD.

ACO	Test	Full-F1 — TR-RGP	Full-F1 — PH-RGP
AS	TSP100-U	-0.525 [-0.876, -0.265]	-0.068 [-0.230, +0.100]
AS	TSP500-U	-0.994 [-1.580, -0.153]	+0.795 [+0.597, +1.007]
ACS	TSP100-U	-0.006 [-0.179, +0.113]	-0.263 [-0.393, -0.167]
ACS	TSP500-U	+0.304 [-1.843, +2.336]	-2.950 [-4.575, -1.219]
MMAS	TSP100-U	-0.056 [-0.182, +0.144]	-0.828 [-0.970, -0.659]
MMAS	TSP500-U	-0.810 [-2.601, +1.187]	-1.162 [-3.075, +0.731]

TR-RGP. Relative to PH-RGP, Full-F1 has three intervals below zero, two intervals crossing zero, and one AS/TSP500 interval above zero. No context simultaneously shows that Full-F1 is better than both one-tree methods.

2) *31/62-Node Capacity Sensitivity*: This experiment is ongoing. It retrains TR-RGP, PH-RGP, and Full-F1. A one-tree method can use 62 nodes at $B = 62$. The two-tree

Dual versus independently trained single tree

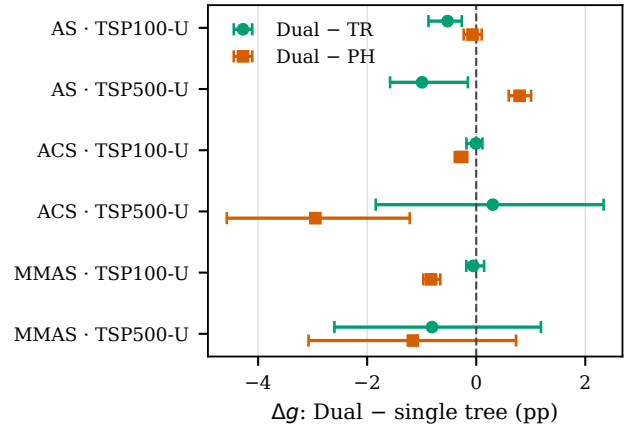


Fig. 7. Two-tree versus independently trained one-tree differences under a total 31-node budget. Error bars are 95% hierarchical bootstrap intervals. Negative values favor the two-tree method.

method shares 62 total nodes at $B = 62$. This design separates the number of semantic roles from total expression capacity. Deleting one tree from an existing Full-F1 individual does not replace this independent training control.

Current answer to RQ3: At a total budget of 31 nodes, the pilot does not show that two trees consistently outperform both one-tree controls. The capacity study is required to determine whether this result is caused by the total node limit.

D. RQ4: Terminal and Function Profiles

The factorial experiment contains Core-F0, Core-F1, Full-F0, and Full-F1. The average Full-minus-Core difference is the terminal main effect. The average F1-minus-F0 difference is the function main effect. The difference in these differences is the interaction.

No Full-minus-Core interval is below zero. Four intervals cross zero. The MMAS TSP100 and TSP500 intervals are above zero and favor Core. No F1-minus-F0 interval is below zero. Five intervals cross zero. The AS TSP500 interval is above zero and favors F0. All six interaction intervals cross zero.

Fig. 9 provides a syntax audit. BaseConf appears in 6/9 transition champions. DistRank and ConstructProg appear in none. Entropy, ACOProg, and Stagnation each appear once. For pheromone trees, EdgeTau appears in 6/9 champions and ColonyFreq appears in 5/9. ACOProg and Stagnation each appear once. Syntactic occurrence is not causal contribution. A terminal can appear in a canceled subexpression.

Answer to RQ4: The pilot does not show a stable benefit from the Full terminal set or the F1 additions. Three contexts instead support a simpler profile. The current Full/F1 design may therefore be over-specified. The Full-F1 main results remain valid, but a confirmatory study should prioritize Core-F0 or Core-F1.

TABLE XVI

CORE/FULL $\times$ F0/F1 FACTORIAL EFFECTS. EACH CELL IS THE MEAN CONTRAST [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS.

ACO	Test	Full – Core	F1 – F0	(Full-F1 – Full-F0) – (Core-F1 – Core-F0)
AS	TSP100-U	-0.213 [-0.534, +0.021]	+0.501 [-0.254, +0.990]	-0.410 [-0.900, +0.406]
AS	TSP500-U	-0.414 [-1.433, +0.844]	+0.951 [+0.066, +2.504]	-1.953 [-5.492, +1.437]
ACS	TSP100-U	-0.030 [-0.122, +0.071]	-0.022 [-0.058, +0.016]	-0.005 [-0.063, +0.057]
ACS	TSP500-U	+0.210 [-1.043, +2.296]	-0.219 [-1.197, +0.373]	+0.074 [-0.383, +0.548]
MMAS	TSP100-U	+0.085 [+0.017, +0.155]	+0.033 [-0.042, +0.143]	-0.004 [-0.215, +0.186]
MMAS	TSP500-U	+0.528 [+0.080, +1.163]	+0.353 [-1.106, +1.365]	+0.549 [-1.514, +2.519]

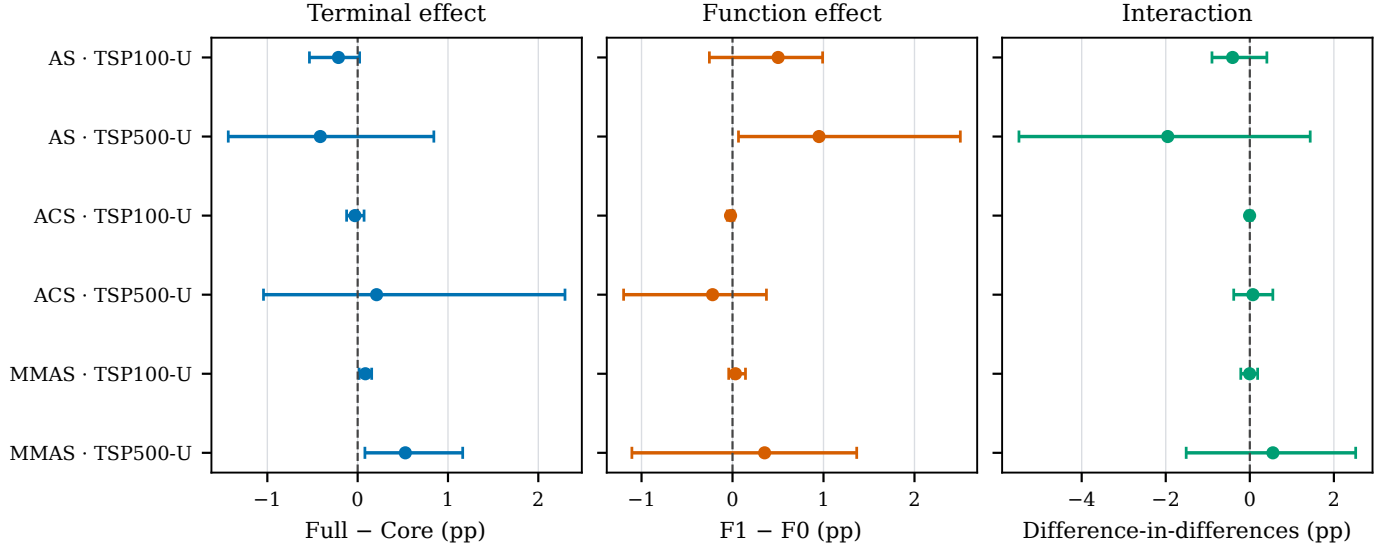


Fig. 8. Terminal main effect, function main effect, and their interaction. Negative values indicate a reduction in reference gap. Error bars are 95% hierarchical bootstrap intervals.

TABLE XVII

POST-TRAINING COMPONENT DELETION AND CROSS-RUN PAIRING INTERVENTIONS FOR FULL-F1. EACH CELL IS THE MEAN DIFFERENCE [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS. NEGATIVE VALUES FAVOR THE ORIGINAL FULL-F1 PAIR.

ACO	Test	Full – drop-TR	Full – drop-PH	Full – shuffle-r1	Full – shuffle-r2
AS	TSP100-U	-0.270 [-0.728, +0.000]	-1.290 [-1.576, -1.058]	-0.214 [-0.349, -0.097]	-0.292 [-0.735, +0.211]
AS	TSP500-U	-1.824 [-5.132, +0.000]	-6.090 [-7.765, -3.058]	-0.666 [-5.459, +2.957]	-0.749 [-5.407, +3.422]
ACS	TSP100-U	-0.273 [-0.321, -0.222]	-0.018 [-0.091, +0.048]	-0.032 [-0.092, +0.039]	-0.007 [-0.147, +0.121]
ACS	TSP500-U	-2.817 [-4.044, -0.854]	-0.341 [-0.659, -0.019]	-0.116 [-0.598, +0.273]	-0.236 [-0.540, +0.110]
MMAS	TSP100-U	-1.534 [-2.102, -1.006]	-0.134 [-0.255, +0.000]	+0.014 [-0.254, +0.239]	+0.026 [-0.135, +0.250]
MMAS	TSP500-U	-4.403 [-4.679, -4.140]	-1.551 [-2.834, +0.000]	-0.027 [-1.769, +2.845]	-0.029 [-2.834, +1.633]

E. RQ5: Joint Coadaptation

The first intervention replaces the transition tree with zero. The second replaces the pheromone tree with zero. All six Full-minus-drop-TR means are negative. Four intervals are strictly below zero. The other two have an upper bound of zero. The transition tree is therefore usually necessary for the selected champions.

Full-minus-drop-PH is also mainly negative. Three intervals are strictly below zero. Two have an upper bound of zero. One crosses zero. The pheromone tree also contributes, but its

evidence is less consistent.

The third intervention shuffles the two trees across independent GP runs. Only the AS/TSP100 r1 interval is below zero among the 12 shuffle intervals. The other 11 cross zero. The current data therefore do not show a stable advantage for the original within-run pairing.

Answer to RQ5: Both components affect the selected Full-F1 individuals. The transition-tree effect is more consistent. The pilot does not show stable pair-specific coadaptation. This post-hoc result does not replace the independently trained one-

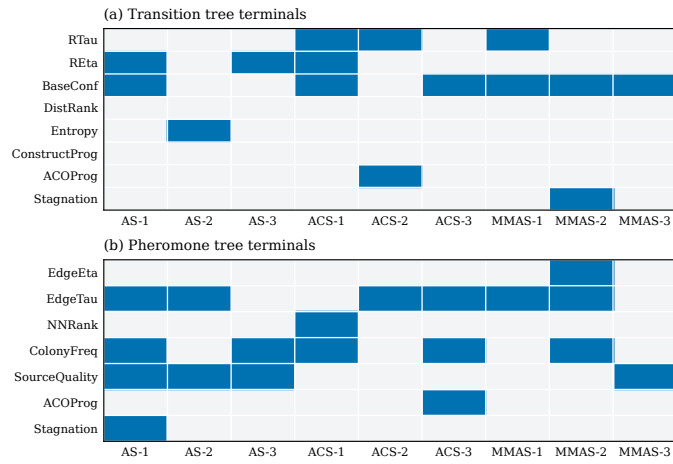


Fig. 9. Syntactic terminal use by the nine Full-F1 champions. Blue indicates that an expression references a terminal at least once. The plot does not test whether a reference remains active after algebraic simplification.

tree controls in RQ3.

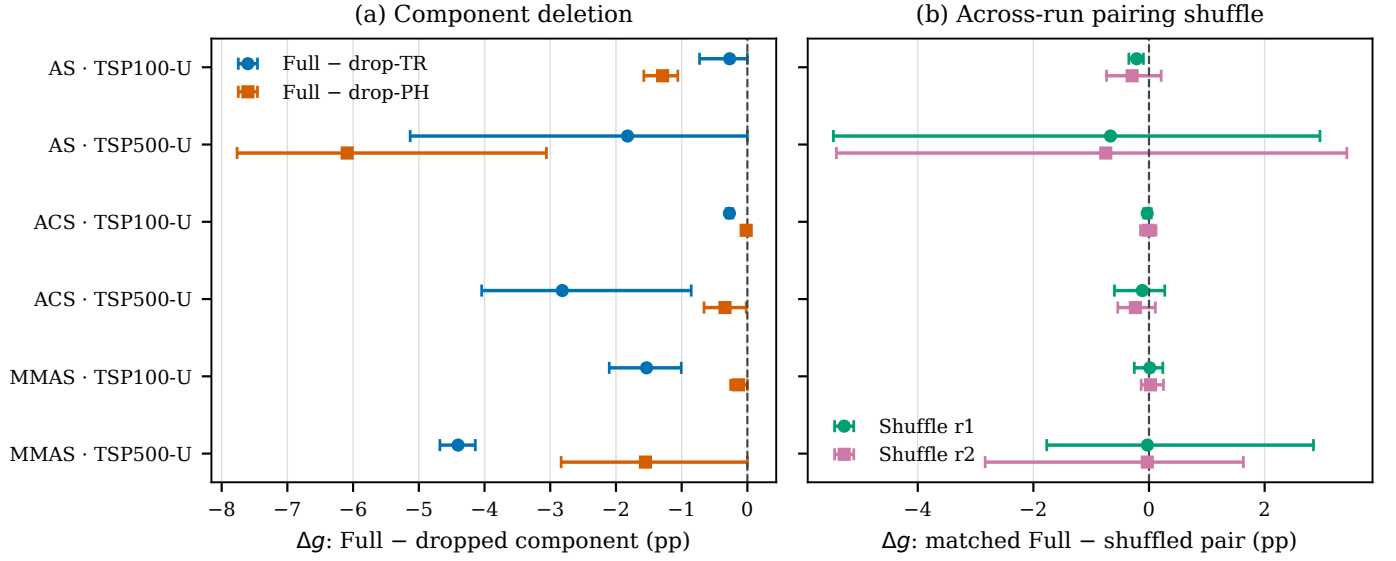


Fig. 10. Post-training tree deletion and cross-run tree-pair shuffling. Negative values favor the original Full-F1 champion. Error bars are 95% hierarchical bootstrap intervals.

F. RQ6: Generalization

The uniform results support scale transfer. Training and validation use only TSP100. TSP50, TSP500, and TSP1000 are unseen sizes. All nine ACO-variant-unseen-size mean differences are negative. The absolute reduction is generally larger at TSP500 and TSP1000.

All nine distribution-shift mean differences in Table XVIII are negative. The cluster reduction ranges from 2.5182 to 6.3245 percentage points. The Gaussian reduction ranges from 2.2175 to 6.0802 percentage points. The TSPLIB reduction ranges from 0.9399 to 3.3419 percentage points.

The TSPLIB win rates are lower than the two synthetic OOD rates. They are 82.5% for AS, 77.8% for ACS, and 93.7% for MMAS. This result indicates more heterogeneous transfer to benchmark instances. The OOD evaluation uses only locked Full-F1 champions. It does not reselect programs.

Answer to RQ6: The pilot supports transfer across uniform problem sizes. All nine OOD means also support distribution transfer. The OOD results are descriptive and do not prove a universal advantage for the residual, terminal, or function design.

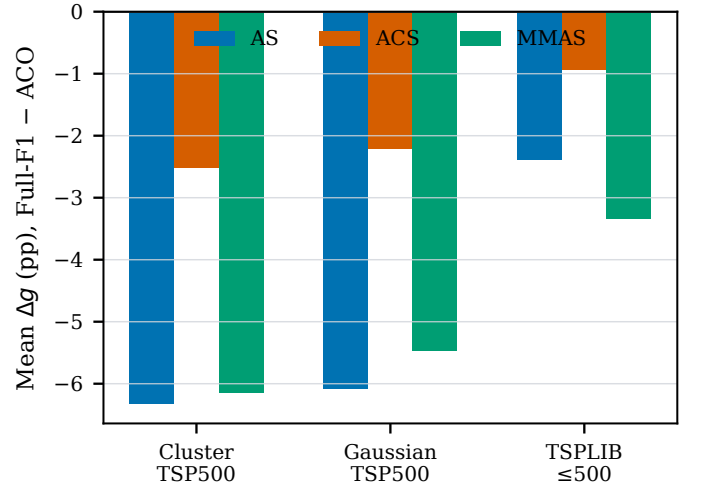


Fig. 11. Descriptive mean differences on cluster, Gaussian, and TSPLIB partitions. Negative values favor Full-F1. The figure does not show confirmatory intervals.

TABLE XVIII

DESCRIPTIVE DISTRIBUTION-SHIFT RESULTS. W/T/L IS THE PERCENTAGE OVER GP-RUN-INSTANCE BLOCKS. THE TABLE IS NOT USED FOR CAUSAL CLAIMS ABOUT TERMINALS OR FUNCTIONS.

ACO	Test	Baseline gap (%)	Full-F1 gap (%)	Δg (pp)	W/T/L
AS	TSP500-C	26.0627	19.7383	-6.3245	100.0/0.0/0.0
AS	TSP500-G	20.0986	14.0184	-6.0802	100.0/0.0/0.0
AS	TSPLIB $\leq$ 500	10.0277	7.6313	-2.3964	82.5/0.0/17.5
ACS	TSP500-C	16.3527	13.8345	-2.5182	90.6/0.0/9.4
ACS	TSP500-G	14.8973	12.6798	-2.2175	88.3/0.0/11.7
ACS	TSPLIB $\leq$ 500	4.5376	3.5976	-0.9399	77.8/0.0/22.2
MMAS	TSP500-C	22.7375	16.5961	-6.1414	100.0/0.0/0.0
MMAS	TSP500-G	17.6609	12.1975	-5.4635	100.0/0.0/0.0
MMAS	TSPLIB $\leq$ 500	8.3755	5.0336	-3.3419	93.7/2.4/4.0

G. RQ7: Computational Efficiency

The fused evaluator gives a 14.626-fold single-GPU speedup over CPU16 on the registered workload. Dual sharding gives a 23.820-fold speedup over CPU16. Its scaling over one GPU is 1.629. Concurrent two-run campaign execution gives a 1.972-fold throughput scaling over one GPU.

Table XIX uses the current three RTX A5000 GPUs. Each champion is timed after an individual warm-up. This experiment measures deployment overhead and is not mixed with the RTX 4000 Ada fused-population throughput benchmark. All three variants have completed TSP50 and TSP100. ACS has also completed TSP500. The seven completed median overheads range from +53.83% to +487.25%. The completed AS and MMAS overheads are larger than the ACS overheads. High fused-training throughput therefore does not imply zero single-program deployment cost. An “Ongoing” cell is not treated as zero and is not included in that range.

Current answer to RQ7: Population-by-instance fusion makes full-budget GP training practical. Independent runs are the preferred multi-GPU unit. The training-throughput ratios apply to the registered RTX 4000 Ada workload. The isolated deployment overheads apply to the current RTX A5000 measurement. Other hardware requires a new benchmark.

VII. LIMITATIONS AND THREATS TO VALIDITY

The main limitation is the number of GP runs. Three runs can reveal major failures. They cannot estimate evolutionary uncertainty with high precision. A confirmatory experiment requires 30 independent GP runs.

Training uses only uniform TSP100. The conclusions apply only to the reported AS, synchronous ACS, and MMAS parameters. Local search is disabled. Distribution shift covers two synthetic distributions and 42 TSPLIB instances. These OOD results are descriptive.

The reference tours are valid. They do not all have attached global optimality certificates. Reference gap can therefore be negative in principle. The metric remains suitable for paired comparison because every method uses the same reference.

GPU search uses FP32. CPU and GPU can follow different stochastic trajectories. The final length calculation is FP64, and the registered numerical non-inferiority test passes. This result does not imply bitwise equivalence.

TABLE XIX

ISOLATED WARM INFERENCE OVERHEAD OF FULL-F1 RELATIVE TO THE ORIGINAL ACO

ACO	TSP50	TSP100	TSP500	TSP1000
AS	+467.11%	+340.88%	Ongoing	Ongoing
ACS	+53.83%	+63.27%	+112.91%	Ongoing
MMAS	+487.25%	+418.58%	Ongoing	Ongoing

RQ1 alone cannot support a causal explanation. This paper reports controlled pilot comparisons for residual integration, tree architecture, terminal and function profiles, and post-hoc coadaptation. Three GP runs are still insufficient for confirmatory causal claims. The 31/62-node capacity comparison is ongoing.

VIII. CONCLUSION

This paper introduced RMTGP-ACO. The method evolves two strongly typed residual programs. One program changes state transition desirability. The other redistributes pheromone deposits. The interfaces preserve useful ACO invariants. The two trees also share a fixed total node budget.

The completed pilot uses three independent GP runs for AS, ACS, and MMAS. Full-F1 has a lower mean reference gap than baseline ACO in every completed uniform test. The result holds from TSP50 to TSP1000. All nine mean distribution-shift differences also favor Full-F1.

The ablations qualify the main result. Residual integration is better for AS, complete replacement is better for ACS, and MMAS is uncertain. The richer Full terminal set and F1 function set do not show stable benefits. Both evolved trees affect the selected champions, but pair-specific coadaptation is not stable. Under a total 31-node budget, the two-tree method does not consistently outperform both one-tree controls. The 31/62-node capacity experiment remains necessary for a capacity-adjusted architecture conclusion.

The fused CUDA evaluator substantially reduces the cost of population training. The partial isolated measurements also show non-negligible deployment overhead for one learned program. All findings remain three-run pilot evidence. They motivate a larger confirmatory experiment with simpler Core profiles and more independent GP runs.

REFERENCES

- [1] M. Dorigo, V. Maniezzo, and A. Coloni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
- [3] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press, 1992.
- [4] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu Enterprises, 2008, with contributions by J. R. Koza.
- [5] B.-C. Lin, Y. Mei, and M. Zhang, "Automated design of state transition rules in ant colony optimization by genetic programming: A comprehensive investigation," *Memetic Computing*, vol. 17, p. 2, 2025.
- [6] J. Tavares and F. B. Pereira, "Designing pheromone update strategies with strongly typed genetic programming," in *Genetic Programming*, ser. Lecture Notes in Computer Science, vol. 6621. Springer, 2011, pp. 85–96.
- [7] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [8] T. Stützle and H. H. Hoos, "MAX–MIN ant system," *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [9] T. Stützle, "ACOTSP, version 1.0." Software package, 2004, version 1.03 used as the algorithmic reference.
- [10] D. J. Montana, "Strongly typed genetic programming," *Evolutionary Computation*, vol. 3, no. 2, pp. 199–230, 1995.
- [11] F.-A. Fortin, F.-M. D. Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.
- [12] G. Reinelt, "TSPLIB: A traveling salesman problem library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [13] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [14] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall/CRC, 1994.
- [16] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [17] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, 2015, pp. 1–6.
- [18] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [19] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proceedings of the Workshop on Machine Learning Systems at NeurIPS*, 2017.